🖐 MediaPipe Gesture Interactive ID-Based AprilTag Sorting

1. Course Overview

The function implemented in this lesson can be simply understood as: **show a number gesture to tell the robot which one to grasp, robot automatically finds the corresponding numbered AprilTag and picks it up**.

The system uses MediaPipe's 21-point hand skeleton keypoint detection algorithm to recognize gesture numbers (1/2/3/4, etc.) in real-time, publishes the gesture ID via ROS topic to the AprilTag sorting node. Based on the received gesture number, the sorting node automatically filters and retrieves the AprilTag target block on the table. After chassis adaptive approach and alignment, the robotic arm executes IK descent grasping and classified unloading, then automatically resets waiting for the next gesture command.

> **Note**
>
> **This tutorial uses the accompanying 3×3cm AprilTag cube wooden blocks.**

---

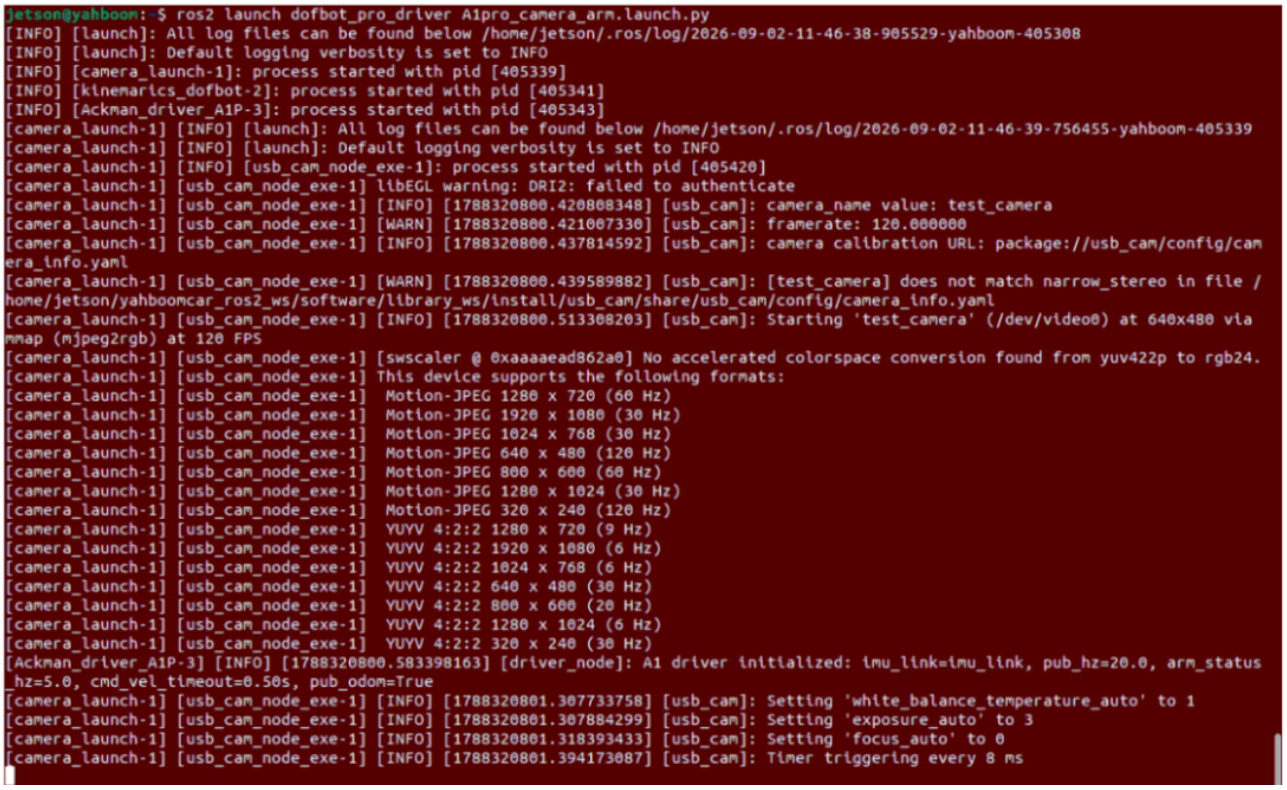
2. Preparation and Running

This case requires opening 4 terminals to start the system sequentially:

2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

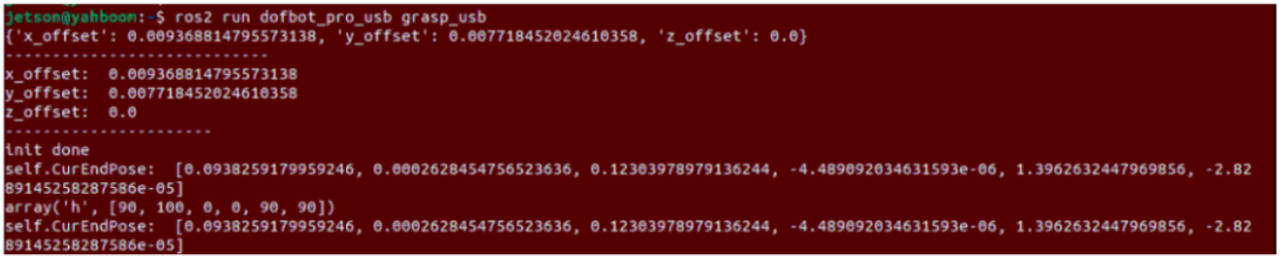


2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```



2.3 Start Gesture ID Sorting Master Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver apriltagID_gesture

# Or USB camera version
ros2 run dofbot_pro_usb apriltagID_gesture
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb apriltagID_gesture
init done
{'x_offset': 0.009368814795573138, 'y_offset': 0.007718452024610358, 'z_offset': 0.0}
----------------------------
x_offset:  0.009368814795573138
y_offset:  0.007718452024610358
z_offset:  0.0
----------------------
Init done.
```

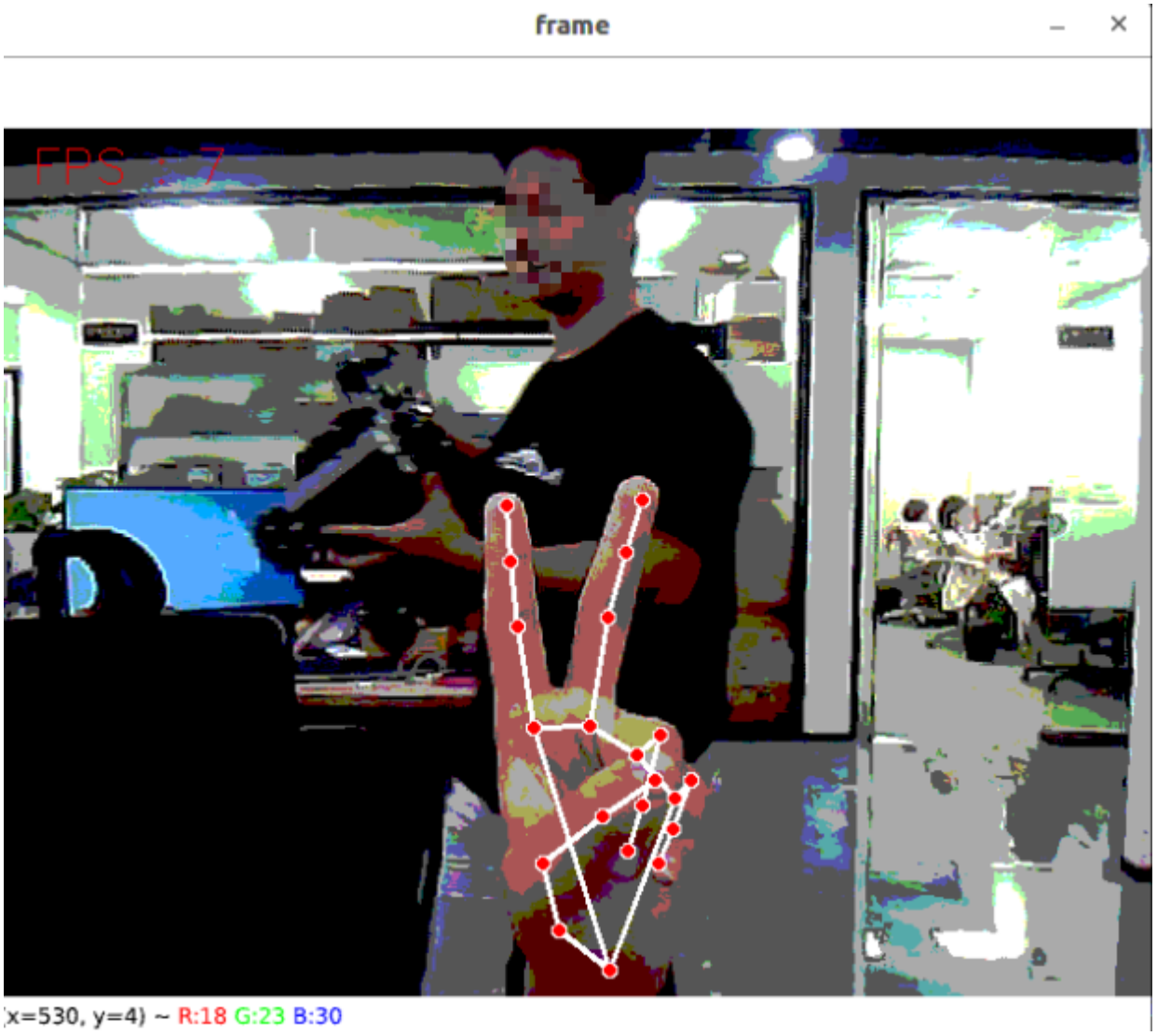
2.4 Start MediaPipe Gesture Recognition Node

Run in **Terminal 4**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver mediapipe_detect

# Or USB camera version
ros2 run dofbot_pro_usb mediapipe_detect
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb mediapipe_detect
/home/jetson/.local/lib/python3.10/site-packages/matplotlib/projections/__init__.py:63:
UserWarning: Unable to import Axes3D. This may be due to multiple versions of Matplotlib
 being installed (e.g. as a system package and as a pip package). As a result, the 3D pr
ojection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple versions of "
init done
----------------------
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```



3. Operation Demo and Interaction Flow

1. **Place materials**: Place multiple AprilTag wooden blocks with different IDs (e.g., ID: 1, 2, 3, 4) flat in the robotic arm's front working area;
2. **Gesture interaction recognition**:
    - Extend your hand in front of the camera (approximately 30~50cm from lens), keep palm open and show number gesture:
        - **Show gesture "1"**: System recognizes Gesture=1, robotic arm locks and grasps ID:1 AprilTag block;
        - **Show gesture "2"**: System recognizes Gesture=2, robotic arm locks and grasps ID:2 AprilTag block;
        - **Show gesture "3"**: System recognizes Gesture=3, robotic arm locks and grasps ID:3 AprilTag block;
        - **Show gesture "4"**: System recognizes Gesture=4, robotic arm locks and grasps ID:4 AprilTag block;
3. **Robust filtering confirmation**:
    - Gesture needs to remain continuously stable for approximately 1 second in the field of view to

prevent false triggering;
    - After confirmation and lock, buzzer sounds confirmation, robotic arm automatically switches from face/gesture interaction pose to desktop patrol pose;
4. **Autonomous approach and grasping**:
    - After retrieving the AprilTag matching the gesture, chassis automatically and smoothly advances to align with the material;
    - Robotic arm adaptively calculates gripper rotation angle and descends to grasp, placing in the classified unloading bin;
5. **Reset and wait for new command**:
    - After grasping, front wheel servo straightens, robotic arm lifts back to gesture recognition pose, sends reset signal to gesture node, ready to show the next gesture.

> **Tip**
>
> When showing gestures, please keep finger joints extended and clear, palm facing the camera, avoid complex human movement interference in the background.

---

4. Core Source Code Analysis

The system works based on a typical ROS 2 asynchronous publish-subscribe architecture:

Node Function	USB Monocular Camera Solution	3D Depth Camera Solution (Nuwa-HP60C)
Gesture Skeleton Detection	`dofbot_pro_usb/mediapipe_detect.py`	`dofbot_pro_driver/mediapipe_detect.py`
Gesture Sorting Master	`dofbot_pro_usb/apriltagID_gesture.py`	`dofbot_pro_driver/apriltagID_gesture.py`
IK Grasping Execution	`dofbot_pro_usb/grasp_usb.py`	`dofbot_pro_driver/grasp_desktop.py`

The main technical implementation comparison:

Comparison Dimension	USB Monocular Solution	3D Depth Camera Solution
Distance Measurement	Pinhole geometry calculates camera frame distance	Hardware depth matrix directly reads tag center true physical depth
Target Approach Working Distance	Chassis PID control advances to approximately 190mm	Chassis PID control advances to approximately 205mm (`GRASP_DIST = 205`)
Spatial Calibration Compensation	Superimpose X, Y 2D plane calibration deviation	Synchronously superimpose X, Y, Z 3D omnidirectional calibration compensation (`offset_value.yaml`)
Gripper Surface Adaptive	Fixed vertical pose grasping	Combine AprilTag corner vectors, adaptively rotate via `compute_joint5` for surface alignment

---

4.1 MediaPipe 21-Point Gesture Skeleton Computation (`mediapipe_detect.py`)

MediaPipe normalizes images and extracts 21 hand 3D keypoints (Landmarks). The program determines finger extension/flexion status by comparing the spatial height of fingertips and knuckles for thumb, index, middle, ring, and pinky, using a continuous 15-frame voting counter (`gesture_counter`) to lock stable gestures:

```
# mediapipe_detect.py -- Skeleton extension detection and continuous frame confirmation
# Iterate through four fingers: if fingertip Y coordinate is less than knuckle Y coordinate, finger is extended (image coordinate Y points down)
for tip in [8, 12, 16, 20]:
    if hand_landmarks.landmark[tip].y < hand_landmarks.landmark[tip - 2].y:
        fingers_up += 1

# Publish topic only after consistent for 15 frames, prevent misoperation
if detected_id == self.last_gesture_id:
    self.gesture_counter += 1
    if self.gesture_counter >= 15:
        msg = Int16()
        msg.data = detected_id
        self.pub_gesture_id.publish(msg)  # Publish GesturetId topic
```

4.2 Gesture-Driven and Target Filtering (`apriltagID_gesture.py`)

1. Topic Listening and Pose Smooth Switching

The sorting node listens to `GesturetId` topic. Once receiving target gesture number (e.g., 2), it first drives the robotic arm to switch to desktop top-down ready pose. After approximately 1 second delay waiting for mechanism to fully stop, it activates AprilTag retrieval:

```
# apriltagID_gesture.py -- Gesture callback and pose switching
def get_GesturetIdCallBack(self, msg):
    self.TargetID = msg.data            # Save gesture target ID (1/2/3...)
    self.pubSix_Arm(self.init_joints)   # Robotic arm switches to desktop patrol pose
    self.start_time = time.time()
    self.count = True                   # Enable delay stability timing
    self.pubPos_flag = False
```

2. Precise Matching Filter and Hardware-Level Ranging (Depth Camera)

In the image callback, detect all AprilTags in the field of view. Only when `tag.tag_id == self.TargetID` is it treated as a valid target. The depth camera version directly extracts the true physical depth at the tag center and calculates the world coordinate absolute position:

```
# apriltagID_gesture.py -- Target matching and physical depth extraction
for tag in tags:
    if tag.tag_id != self.TargetID:
        continue  # Ignore blocks with other irrelevant IDs
```

```
    # Extract tag center physical depth (meters)
    cz = depth_image[int(tag.center[1]), int(tag.center[0])] / 1000.0
    if cz != 0:
        pose = self.compute_pose(tag.center[0], tag.center[1], cz)
        dist_detect = math.hypot(pose[0], pose[1]) * 1000.0  # Horizontal mm distance

        # Chassis PID coordination adjusts material distance to optimal grasping point
(205mm)
        if abs(dist_detect - self.cen_dist) > 5.0 and self.adjust_dist:
            self.move_dist(dist_detect, tag.center[0])
```

3. Gripper Rotation Alignment and Closed-Loop Reset

After chassis stops, use tag corner slope vector to solve joint 5 pose (`compute_joint5`), publish 3D coordinate command to trigger grasping; after grasping completes, execute front wheel servo straightening action and publish reset signal to `reset_gesture` topic, notifying gesture node to restart listening for new hand interaction.

4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
    - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
    - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
    - Gripper deviates right **increase value** (correct left)
    - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:
    - Gripper too high in air and can't grasp **decrease value** (probe deeper)
    - Gripper hits or scrapes table **increase value** (raise higher)